

virtual - методът може да бъде презаписан (override-ван) в наследниците на класа и извикването му при runtime да избере правилната имплементация.

- Позволява динамично свързване
- За дефиниране на абстрактни интерфейси и полиморфизъм

Статично свързване - компилаторът решава коя функция да се извика

Динамично свързване - изборът се прави ~~по време~~ при изпълнение чрез vtable; позволява по-вероятно според runtime типа.

Виртуални функции

- Декларира се virtual в базовия клас
- В наследника се имплементира същата функция с override
- чисто виртуална функция (=0) - прави класа абстрактен
- Деструкторът на базовия клас трябва да е virtual, инак ресурсите не се освобождават правилно

Override и final → маркира метод или клас, че не могат да бъдат презаписвани
↳ показва, че функцията е предназначена да презапише виртуална функция от базовия клас и компилаторът проверява за коректност (ако не съвпадат → грешка)

- Без override и неправилна функция, компилаторът ще е въведе, няма да има грешка, но методът няма да бъде извикан полиморфно.

Чисти виртуални функции (=0)

- Клас с поне една (=0) → абстрактен и не може да се инстанцира
- Клас само с (=0) → интерфейс

Виртуални деструктори

- Ако имаме полиморфни класове и прием чрез Base*, базовият деструктор трябва да е виртуален, инак undefined behavior (ресурсите не се освобождават правилно)

Virtual table

- Всеки клас с виртуални функции, компилаторът генерира vtable.
- ~~В~~ Всеки обект съдържа указател (vptr) към vtable на реалния си клас
- При виртуално извикване: следва vptr → води до адрес на функция в vtable → извиква я

- Един vtable на клас (споделен), един vptr на обект

Open/Close principle (O in SOLID)

- Класовете са отворени за разширяване, затворени за промяна
- При всеки нов тип имаме нов if
=> полиморфизъм

Статичен полиморфизъм - изборът се прави по време на компилация
(няма vtable)

- по-бърз е
- може да се постигне с: `templates`,
`std::variants`